SECTION 1 — COMPLETE COMBINATIONAL NOTES (F215 Digital Design)

> **Scope:** everything from the start of the course up to (not including) Sequential Circuits. **Built from:** Lec-6 → Lec-15 slides, the textbook (Mano & Ciletti), the course handout, Tut-3 → Tut-5, Quiz 2 (both sets), Midsem 2024-25, Midsem 2025-26, and the combinational questions on the Comprehensive papers. **Convention used everywhere below:** the first-named variable is the MSB. `'` means complement (`A'` = NOT A). `⊕` = XOR, `⊙` = XNOR. **Tag:** every examiner trap is marked ⚠️ **EXAM TRICK:**

---

0. MASTER TOPIC CHECKLIST

- **Module 1: Working in binary**
  - ☐ Number systems (binary, octal, hex, base-r), conversions (integer + fraction)
  - ☐ Signed numbers: sign-magnitude, 1's complement, 2's complement, sign extension
  - ☐ Binary addition/subtraction, unsigned vs signed overflow, the **"bit-width detective"** question
  - ☐ Binary codes: BCD (8421), Excess-3, Gray code, parity (generator/checker)
  - ☐ Boolean algebra: postulates, theorems, De Morgan, absorption, consensus, duality
  - ☐ Logic gates, XOR/XNOR properties, universal gates
  - ☐ Minterms, maxterms, canonical SOP/POS, standard forms, converting between them
  - ☐ K-maps: 2/3/4/5 variables, PIs, EPIs, don't-cares, minimal SOP (MSP), minimal POS (MPS), cyclic maps
  - ☐ Quine–McCluskey tabular method + PI chart
  - ☐ NAND-only / NOR-only realisation, bubble pushing, 2-input-gate-only designs
- **Module 2: Combinational building blocks**
  - ☐ Half adder, full adder, FA from 2 HAs
  - ☐ Ripple-carry adder and its delay (`2n+1`)
  - ☐ Half subtractor, full subtractor, FS from 2 HSs (and the classic wiring bug)
  - ☐ 4-bit adder/subtractor (XOR = programmable inverter), overflow flag `V = Cn-1 ⊕ Cn`
  - ☐ Carry-lookahead adder (G/P), borrow-lookahead, scaling (O(n), O(1), O(n^2), O(log n))
  - ☐ BCD (decimal) adder
  - ☐ Code converters: BCD→Excess-3, Binary↔Gray
  - ☐ Magnitude comparator
  - ☐ Binary multipliers (2×2, J×K), multiply by a constant
  - ☐ Decoders (active-high/active-low, enable, hierarchical), implementing functions with decoders
  - ☐ Encoders, priority encoders, decoder→encoder code converters
  - ☐ Multiplexers (2:1, 4:1, 8:1, enable, dual mux), implementing functions with muxes (3 methods), cascading muxes
  - ☐ Demultiplexers
  - ☐ Using adders/subtractors **as gates** (component-restricted design)
  - ☐ PLDs: ROM, PLA, PAL, programming tables, minimum-size PLA
- **Module 4 (combinational part): Verilog**
  - ☐ Gate-level (structural) modelling, module instantiation (named vs positional ports)

- ☐ Dataflow (`assign`), operators, precedence
- ☐ Behavioural (`always`, `initial`), blocking vs non-blocking
- ☐ `wire` vs `reg`, testbenches, system tasks (`$display`, `$monitor`, `$finish`, `$stop`)

---

1. NUMBER SYSTEMS & CONVERSION

1.1 Positional value

- A number in base `r` with digits `d_i`: `Value = Σ d_i · r^i` (i = 0 at the digit left of the point; negative `i` after the point).
- Binary weights (right → left): `1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096`.
- `n` bits → `2^n` distinct patterns → unsigned range `0 … 2^n − 1`.
- To represent `N` distinct things you need `⌈log2 N⌉` bits.
  - 8 LEDs → 3 bits; 10 decimal digits → 4 bits; 6 states → 3 bits; 40 cars → 6 bits.

1.2 Binary ↔ Octal ↔ Hex (no arithmetic)

- Octal digit = 3 bits; Hex digit = 4 bits (`A=10, B=11, C=12, D=13, E=14, F=15`).
- **Always group starting from the binary point** — leftward for the integer part, rightward for the fraction; pad the far group with 0s.

```
1011010.11 (binary)
Hex   :  0101 1010 . 1100   →  5A.C
Octal:  001 011 010 . 110  →  132.6
```

- ⚠️ EXAM TRICK: grouping from the left end of the integer instead of from the point gives a totally different answer. Pad on the **outside**, never in the middle.

1.3 Decimal → binary

- **Integer part:** divide by 2 repeatedly; the remainders are the bits, **LSB first** (read bottom-up).
- **Fraction part:** multiply by 2 repeatedly; the integer that pops out is the next bit after the point, **MSB first** (read top-down).

```
Integer 41:                        Fraction 0.6875:
  41 / 2 = 20  r 1  ← LSB            0.6875 × 2 = 1.375  → 1  ← first bit after point
  20 / 2 = 10  r 0                   0.375  × 2 = 0.75   → 0
  10 / 2 =  5  r 0                   0.75   × 2 = 1.5    → 1
   5 / 2 =  2  r 1                   0.5    × 2 = 1.0    → 1  (stops at 0)
   2 / 2 =  1  r 0
   1 / 2 =  0  r 1  ← MSB          41.6875 = 101001.1011
```

- Fractions may never terminate (e.g. $0.1_{10} = 0.000110011\ldots_2$) → stop at the precision asked for.
- Any base `r`: divide by `r` (integer part), multiply by `r` (fraction part).
- Base `r` → decimal: always `Σ digit × weight`.

---

2. SIGNED NUMBERS

2.1 Three representations (4-bit examples)

Value	Sign-magnitude	1's complement	2's complement
+7	0111	0111	0111
+1	0001	0001	0001
+0	0000	0000	0000
−0	1000	1111	— (no −0)
−1	1001	1110	1111
−7	1111	1000	1001
−8	—	—	1000
Range	−7…+7	−7…+7	**−8…+7**

- **Sign-magnitude:** MSB = sign, rest = magnitude. Two zeros.
- **1's complement:** negate by inverting every bit. Two zeros.
- **2's complement:** negate = invert all bits **+ 1**. One zero. Used by all hardware and this course.
  - MSB has weight `-2^(n-1)`, the other bits are normal: `1011 = -8 + 0 + 2 + 1 = -5`.
  - n-bit range: `-2^(n-1) … +2^(n-1) - 1` (one extra negative number).
  - 3-bit: −4…+3; 4-bit: −8…+7; 5-bit: −16…+15; 6-bit: −32…+31; 8-bit: −128…+127.
- **Fast negate trick:** copy bits from the right up to and including the first `1`, then invert everything to its left.
  - `0110100 → 1001100`.

2.2 Sign extension

- To widen a 2's-complement number, **copy the MSB** into the new high bits.
  - `-3 = 1101` (4-bit) `= 11111101` (8-bit). Padding with 0s would give +13.
- ⚠️ EXAM TRICK: the most negative number cannot be negated in the same width: `-(1000) = 0111 + 1 = 1000` (−8 again). That is an overflow.

---

3. BINARY ARITHMETIC & OVERFLOW (very high exam weight)

3.1 Subtraction by complement

- `A - B = A + (2's complement of B) = A + B' + 1`.
- So one adder does both jobs: invert B and force the carry-in to 1 (see §13).

3.2 When is the answer wrong?

Situation	Answer is WRONG when…	Why
Unsigned **add**	carry-out `Cn = 1`	true sum needed n+1 bits
Unsigned **subtract** (`A + B' + 1`)	`Cn = 0` (means A < B)	carry = NOT borrow; output is the 2's complement of (B − A)
Signed (2's comp) add **or** subtract	`V = Cn-1 ⊕ Cn = 1`	carry *into* the sign bit ≠ carry *out of* it

- Signed overflow only happens when the two numbers **actually added** have the **same sign** and the result has the opposite sign.
- A positive plus a negative number can **never** overflow: the result lies between the two operands.

- ⚠️ EXAM TRICK: in subtraction mode the numbers "actually added" are A and **B' + 1**. "−3 − 3" is internally "(−3) + (−3)", two negatives, so it can overflow. "2 − 1" is "2 + (−1)", mixed signs, so it cannot.
- ⚠️ EXAM TRICK: the same `V = Cn−1 ⊕ Cn` gate works in both add and subtract mode with **no change**, because the adder only ever sees an addition (Tut-3 Q2c).

3.3 The "bit-width detective" (Quiz 2 Q5, Tut-3 Q2): a recurring exam pattern

- **Given:** a log of additions, some shown correctly and some wrong. **Find:** the adder width.
- **Rule:** a wrong output = (true result) mod `2^n`, re-read as signed if the circuit is signed.
- **Method:**
  - A correct row gives a **lower bound**: that value must fit in `n` bits.
  - A wrong row gives the **modulus**: (true − shown) must be a multiple of `2^n`.
  - Intersect the two.
- **Worked (Quiz 2, Set A, unsigned):**

Prev	Steps	Shown	True	Wrong?	Deduction
20	15	35	35	no	35 fits → n ≥ 6
50	13	63	63	no	63 fits → **n ≥ 6**
50	14	0	64	**yes**	64 − 0 = 64 is a multiple of 2^n → n ≤ 6
45	25	6	70	**yes**	70 − 6 = 64 ✓
10	5	15	15	no	—

- **Answer: n = 6** (2^6 = 64).
- **Worked (Quiz 2, Set B):** 20+12 = 32 → 0 (wrong); 15+25 = 40 → 8 (wrong, 40−8 = 32); 14+14 = 28 correct ⇒ n ≥ 5 ⇒ **n = 5**.
- **Signed version (Tut-3 Q2):** 2+2 shown −4, 3+1 shown −4, −3−3 shown 2, −4−3 shown 1; 1+1 = 2 and 2−1 = 1 correct ⇒ only a **3-bit** 2's-complement adder (range −4…+3) fits all rows. Full table in Section 2, Tut-3 Q2.
- ⚠️ EXAM TRICK: always check **every** row against your chosen n, including the correct ones. A width that explains the wrong rows but would also break a "correct" row is wrong.

---

4. BINARY CODES

4.1 BCD (8421)

- Each decimal digit is written as its own 4-bit binary: `59 → 0101 1001`.
- Codes `1010 … 1111` (10–15) never occur → **don't-cares** **`d(10,11,12,13,14,15)`** in any K-map whose input is a BCD digit.
- ⚠️ EXAM TRICK: BCD ≠ binary. 59_{10} in binary is `111011`, but in BCD it is `0101 1001`.

4.2 Excess-3

- Digit `n` → 4-bit binary of `(n + 3)`: `0 → 0011`, `5 → 1000`, `9 → 1100`.
- **Self-complementing:** inverting all four bits of the code for `n` gives the code for `9 − n` (useful for decimal subtraction).

4.3 Gray code

- Consecutive values differ in **exactly one bit**, including the wrap-around from last to first.
- Used in rotary/angle encoders: a reading taken mid-transition is at worst off by one position.
- **Not weighted**, so convert to binary before doing arithmetic.

Dec	Binary	Gray		Dec	Binary	Gray
0	000	000		4	100	110
1	001	001		5	101	111
2	010	011		6	110	101
3	011	010		7	111	100

- **Binary → Gray:** `g_msb = b_msb`, `g_i = b_(i+1) ⊕ b_i` (parallel XORs, no chain).
- **Gray → Binary:** `b_msb = g_msb`, `b_i = b_(i+1) ⊕ g_i` (a **cascade**: each bit needs the binary bit above it).

```
Binary → Gray (3-bit)                  Gray → Binary (3-bit)
b2 ───────────────────── g2            g2 ──────────●──────────────── b2
b2 ─┐                                               │
    XOR ──────────────── g1            g1 ────────XOR──●──────────── b1
b1 ─┘                                                  │
b1 ─┐                                  g0 ───────────XOR──────────── b0
    XOR ──────────────── g0
b0 ─┘
```

- **Reflect-and-prefix rule for building an n-bit Gray code:** write the (n−1)-bit list, then the same list mirrored; prefix 0 to the first half and 1 to the second half.
- ⚠️ EXAM TRICK (Gray inputs in a K-map, Midsem 2025 Q1, Tut-4 Q2): list the truth-table rows in the **physical (Gray) order** given by the problem, but place each row in the K-map **by its binary value** (its minterm number). E.g. angle sector 3 has code `010`, so its output goes into **cell m2**, not m3. Mixing the two orders is the #1 error.

4.4 Parity

- **Even-parity generator** for data `d3 d2 d1 d0`: `P = d3 ⊕ d2 ⊕ d1 ⊕ d0`. P = 1 when the data has an odd number of 1s, so the total becomes even.
- **Odd-parity generator:** `P = (d3 ⊕ d2 ⊕ d1 ⊕ d0)'`.
- **Checker:** XOR all received bits including P. Result 0 means OK; 1 means error (for even parity).
- Detects **any odd number** of flipped bits; **misses** any even number (two flips cancel).
- ⚠️ EXAM TRICK: a parity/XOR function on a K-map is a **checkerboard**: no two 1s are adjacent, so every minterm is its own EPI and **no K-map simplification is possible** (Tut-4 Q2b: 8 minterms × 4 literals = 32 literals). Use XOR gates instead.

```
Even-parity generator (4 data bits)        Checker (4 data + P)
d3 ─┐                                      d3 ─┐
    XOR─┐                                      XOR─┐
d2 ─┘   │                                  d2 ─┘   XOR──┐
        XOR── P                            d1 ─┐   │    XOR── E (1 = error)
d1 ─┐   │                                      XOR─┘    │
    XOR─┘                                  d0 ─┘        │
d0 ─┘                                      P  ──────────┘
```

---

5. BOOLEAN ALGEBRA

5.1 Postulates and theorems

Law	OR form	AND form (dual)
Identity	`x + 0 = x`	`x · 1 = x`
Null / dominance	`x + 1 = 1`	`x · 0 = 0`
Idempotent	`x + x = x`	`x · x = x`
Complement	`x + x' = 1`	`x · x' = 0`
Involution	`(x')' = x`	
Commutative	`x + y = y + x`	`xy = yx`
Associative	`x + (y + z) = (x + y) + z`	`x(yz) = (xy)z`
Distributive	`x(y + z) = xy + xz`	`x + yz = (x + y)(x + z)` ← not true in ordinary algebra!
Absorption	`x + xy = x`	`x(x + y) = x`
Redundant literal	`x + x'y = x + y`	`x(x' + y) = xy`
De Morgan	`(x + y)' = x'y'`	`(xy)' = x' + y'`
Consensus	`xy + x'z + yz = xy + x'z`	`(x+y)(x'+z)(y+z) = (x+y)(x'+z)`

- **Precedence:** parentheses > NOT > AND > OR. `AB' + C` means `(A·(B')) + C`.
- **Duality:** swap `+ ↔ ·` and `0 ↔ 1` (do not complement the variables). If an identity holds, so does its dual.
- **Generalised De Morgan:** break the bar, flip the operator: `(A + B + C)' = A'B'C'`, `(ABC)' = A' + B' + C'`.
- **Complement of a function:** take the dual and complement each literal: `F = AB' + C` → `F' = (A' + B)C'`.

5.2 Why the "tricky" laws are true

- `x + x'y = x + y`: the `x'y` term only matters when `x = 0`, and then `x' = 1`, so it is just `y`.
- `x + yz = (x + y)(x + z)`: expand the right side: `x + xz + xy + yz = x(1 + z + y) + yz = x + yz`.
- Consensus `xy + x'z + yz`: the term `yz` needs y = z = 1; then either `x = 1` (so `xy` = 1) or `x = 0` (so `x'z` = 1). `yz` is always already covered, so it is redundant.

5.3 Worked simplifications

- `pqr + q + s = q(pr + 1) + s = q + s` (Tut-3 Q3a: **p and r have no effect**).
- `X'Y + (X⊕Y)'·Bin = X'Y + (XY + X'Y')Bin = X'Y + X'Bin + YBin` (full-subtractor borrow, proved in Tut-3 Q1b).
- `AB + A'C + BC = AB + A'C` (consensus).
- `(A + B)(A + C) = A + BC` (second distributive law).
- ⚠️ EXAM TRICK: `(AB)' = A' + B'`, **not** `A'B'`. Breaking a bar always flips AND ↔ OR.
- ⚠️ EXAM TRICK: before drawing any circuit, look for absorption. Verilog "mystery module" questions often hide a function that collapses to fewer inputs (Tut-3 Q3a).

---

6. LOGIC GATES

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

```
AND  A ─┐‾‾\        OR   A ─\‾‾\         NOT  A ─|>o─ A'
        │   )── AB          )   >── A+B
     B ─┘__/             B ─/__/
NAND ──[AND]o──  (AB)'   NOR ──[OR]o── (A+B)'
XOR  ─))─>  A⊕B = A'B + AB'       XNOR ─))─>o  (A⊕B)' = AB + A'B'
```

- **AND** = 1 only if all inputs are 1 (any 0 forces 0). **OR** = 1 if any input is 1.
- **Multi-input XOR** = 1 iff an **odd** number of inputs are 1 (parity). **XNOR** = 1 iff the inputs are equal (1-bit equality comparator).
- **Controlling values (used constantly in design questions):**
  - `A · 1 = A`, `A · 0 = 0` → an AND gate is an **enable/block** switch.
  - `A + 0 = A`, `A + 1 = 1`.
  - `A ⊕ 0 = A`, `A ⊕ 1 = A'` → an XOR gate is a **programmable inverter** (basis of the adder/subtractor).
- XOR identities: `A ⊕ A = 0`, `A ⊕ A' = 1`, `(A ⊕ B)' = A' ⊕ B = A ⊕ B'`, XOR is associative and commutative.

6.1 Universal gates

- NAND alone (or NOR alone) can build every function.

Build	From NAND	From NOR
NOT	1 (tie inputs)	1 (tie inputs)
AND	2 (NAND + NOT)	3 (`A'` NOR `B'`)
OR	3 (`A'` NAND `B'`); **1 if complements are free**	2 (NOR + NOT)
XOR	4	5
XNOR	5	4

```
XOR from 4 NANDs                          XNOR from 4 NORs (same shape, dual)

A ──●──────────────[NAND]── p ─┐          A ──●────────────[NOR]── p ──┐
    │               ▲          │              │             ▲          │
    └──[NAND]── t ──┤          [NAND]── A⊕B   └──[NOR]── t ─┤          [NOR]── A⊙B
    ┌──[     ]      │          │              ┌──[    ]     │          │
B ──●──────────────[NAND]── q ─┘          B ──●────────────[NOR]── q ──┘

t = (AB)'                                 t = (A+B)'
p = (A·t)'    q = (B·t)'                  p = (A+t)'    q = (B+t)'
out = (p·q)' = A⊕B                        out = (p+q)' = A⊙B
(t feeds BOTH second-level gates)
```

7. MINTERMS, MAXTERMS, CANONICAL FORMS

7.1 Definitions (3 variables A, B, C; A = MSB)

Row	A B C	Minterm `m_i` (=1 only on this row)	Maxterm `M_i` (=0 only on this row)
0	0 0 0	`A'B'C'`	`A + B + C`
1	0 0 1	`A'B'C`	`A + B + C'`
2	0 1 0	`A'BC'`	`A + B' + C`
3	0 1 1	`A'BC`	`A + B' + C'`
4	1 0 0	`AB'C'`	`A' + B + C`
5	1 0 1	`AB'C`	`A' + B + C'`
6	1 1 0	`ABC'`	`A' + B' + C`
7	1 1 1	`ABC`	`A' + B' + C'`

- **Minterm:** AND of all variables; a variable that is **0** in that row is primed.
- **Maxterm:** OR of all variables; a variable that is **1** in that row is primed.
- `M_i = (m_i)'`.
- **Canonical SOP** = `Σm(rows where F = 1)`; **canonical POS** = `ΠM(rows where F = 0)`. The two index lists are complementary.
  - `F = Σm(1,3,5) = ΠM(0,2,4,6,7)`; `F' = Σm(0,2,4,6,7)`.
- **Expanding a non-canonical term:** each missing variable doubles the minterms covered: `AB' = AB'C + AB'C' = m5 + m4`.
- **Standard forms:** SOP = a layer of ANDs feeding one OR; POS = a layer of ORs feeding one AND. Both are **2-level** (at most 2 gate delays, ignoring inverters).

7.2 Converting a POS expression to a list of maxterms (Tut-5 Q1b, Q2a)

- `H = (A + B + C)(A + B' + C')(A' + B + C')`:
  - `A + B + C` = 0 at `000` → `M0`.
  - `A + B' + C'` = 0 at `011` → `M3`.
  - `A' + B + C'` = 0 at `101` → `M5`.
  - `H = ΠM(0,3,5) = Σm(1,2,4,6,7)`.
- A term with a missing variable is 0 on several rows: `B` alone (as a POS factor) is 0 whenever B = 0 → `M0·M1·M4·M5`.
- ⚠️ EXAM TRICK: for a maxterm, write the row that makes the OR **zero**: an unprimed variable must be 0, a primed variable must be 1. Students often write the opposite.

---

8. KARNAUGH MAPS

8.1 Layouts (Gray order on both axes: 00, 01, 11, 10)

```
2-var          3-var                          4-var (cell = minterm number)
  B 0  1         BC 00  01  11  10                CD  00  01  11  10
A                A                             AB
0  | 0 | 1 |     0  | 0 | 1 | 3 | 2 |          00  | 0 | 1 | 3 | 2 |
1  | 2 | 3 |     1  | 4 | 5 | 7 | 6 |          01  | 4 | 5 | 7 | 6 |
                                               11  | 12 | 13 | 15 | 14 |
                                               10  | 8 | 9 | 11 | 10 |
```

- **Memorise the 4-var numbering**: rows go 00, 01, **11, 10**, so row 3 holds 12–15 and row 4 holds 8–11; columns 3 and 4 are swapped too (…3, 2 / …7, 6 / …15, 14 / …11, 10).

8.2 Grouping rules

- Two minterms that differ in exactly one variable merge, and that variable disappears: `AB'C + ABC = AC`.
- A group of `2^k` cells removes `k` variables.
- A legal group is a **rectangle of 1, 2, 4, 8 or 16 cells**.
  - It may **wrap** around the left/right edges and the top/bottom edges.
  - The **four corners** form one group (`B'D'` in a 4-var map).
- A group may contain only 1s and any don't-cares you choose to include.
- Make groups **as large as possible** and use **as few as possible**.
- **Reading a group:** variables that stay constant across the group survive (0 → primed, 1 → plain); variables that change vanish.

```
Wrap & corner examples (4-var)
  CD  00 01 11 10            CD  00 01 11 10
AB                         AB
00    1  0  0  1   ← corners   00   0  1  1  0
01    0  0  0  0               01   0  1  1  0  ← centre square = BD
11    0  0  0  0               11   0  1  1  0
10    1  0  0  1   = B'D'      10   0  1  1  0  ← column pair 01,11 = D
```

8.3 Prime implicants, essential prime implicants

- **Implicant:** any legal group of 1s (and Xs).
- **Prime implicant (PI):** a group that **cannot be enlarged** any further.
- **Essential PI (EPI):** a PI that covers at least one **1** that **no other PI** covers. It must appear in every minimal answer.
- **Procedure (always the same):**
  1. Circle **all** PIs.
  2. Mark the EPIs (look for 1-cells with only one PI through them).
  3. Cover the remaining 1s with the fewest, largest PIs.
- **Cyclic map:** no EPIs at all. There are several equally minimal answers; list all of them if asked (Tut-4 Q2c).
- ⚠️ EXAM TRICK: in "list the PIs" questions (Quiz 2 Q2, Midsem 2024 Q3) marks are deducted for **extra** terms and for wrongly ticking EPI. A PI that only re-covers 1s already covered by EPIs is still a PI, so list it if the question asks for all PIs, but do not tick it as essential.

8.4 Don't-cares (X)

- Treat an X as 1 **only** if that makes a group bigger. Each X is decided independently.
- Never form a group made only of Xs.
- An X never makes a PI essential: essentiality is judged on the **1s** only.
- Where do Xs come from?
  - Impossible inputs: BCD codes 10–15, unused states, input combinations the problem says never occur (e.g. F = T = 1 in the turnstile).
  - Outputs nobody looks at: land-facing angles (Midsem 2025), invalid-parity inputs when "the downstream circuit is turned off" (Tut-4 Q2).

8.5 POS from a K-map

- Group the **0s** to get a minimal SOP of `F'`.
- Apply De Morgan to each term: `F' = AB + CD + BD'` → `F = (A' + B')(C' + D')(B' + D)` (Lec-10 example, F = Σ(0,1,2,5,8,9,10)).
- ⚠️ EXAM TRICK (Tut-4 Q1): with **no don't-cares**, the MSP and MPS always describe the **same function**. With **don't-cares**, the MSP and MPS may treat the Xs differently, so they can be

different functions (both correct, since they agree on every specified cell). Never assume MSP = MPS when there are Xs.

8.6 Worked 4-variable example with don't-cares (Quiz 2 Q2, Set A)

- K-map given (A B rows, C D columns):

AB \ CD	00	01	11	10
00	0	0	X	1
01	1	0	1	1
11	1	0	0	X
10	0	0	1	0

- 1s: m2, m4, m6, m7, m11, m12. Xs: m3, m14.
- PIs:
  - `BD'` = {4, 6, 12, 14}: the only PI through m4 and m12 → **EPI**.
  - `A'C` = {2, 3, 6, 7}: the only PI through m2 and m7 → **EPI**.
  - `B'CD` = {3, 11}: the only PI through m11 → **EPI**.
- **MSP:** **`F = BD' + A'C + B'CD`**.
- Set B differs only in row 10 (`1 0 0 0`, so m8 = 1 and m11 = 0) → `AC'D'` = {8, 12} replaces `B'CD`: **`F = BD' + A'C + AC'D'`**.

8.7 Truth table given in scrambled order (Quiz 2 Q1)

- The rows were listed "in the order the tests happened to run", not in binary order.
- Method: for each row, compute the minterm number from A B C and put F in **that** cell. Ignore the listing order completely.
- Set A result: A = 0 row all 1s, A = 1 row `0 0 0 1` (only m6) → `F = A' + BC'`.
- Set B: A = 1 row `0 0 1 0` (only m7) → `F = A' + BC`.
- ⚠️ EXAM TRICK: these questions are marked all-or-none. Double-check each cell, especially the swapped columns 11 and 10.

8.8 Five-variable K-map

- Draw two 4-variable maps: one for `A = 0`, one for `A = 1`.
- A cell is adjacent to its 4 neighbours in its own map **plus** the same cell in the other map.
- A group sitting in the same place in both maps loses `A`.

```
        A = 0                           A = 1
  DE  00 01 11 10                 DE  00 01 11 10
BC                              BC
00    0  1  3  2                00   16 17 19 18
01    4  5  7  6                01   20 21 23 22
11   12 13 15 14                11   28 29 31 30
10    8  9 11 10                10   24 25 27 26
(cell k in the left map is adjacent to cell k+16 in the right map)
```

---

9. QUINE–McCLUSKEY (TABULAR) METHOD

- **Why:** the same merging as a K-map, but done on bit strings, so it scales to any number of variables and is mechanical.
- **Steps:**
  1. List every minterm **and** don't-care in binary, grouped by the number of 1s.

2. Compare each term with every term in the **next** group. If they differ in exactly one bit, write the merged term with `-` in that position and tick both parents.
3. Only merge terms whose dashes are in the **same positions**.
4. Repeat on the new column until nothing merges. **Unticked terms = prime implicants.**
5. Build the **PI chart**: rows = PIs, columns = **required minterms only** (leave out don't-cares). A column with a single × marks an essential PI.
6. Cover the remaining columns with the fewest PIs.

- Dash string → literals: `-` = variable gone, `1` = plain, `0` = primed. `0-01 = A'C'D`.

9.1 Worked example (Comprehensive 2025 Q6): `F = Σm(1,3,4,5,9,10,11) + d(6,8)`

Step 1–2 (column I → II):

#1s	Minterm	Bits		Pair	Merged
1	1	0001 ✓		(1,3)	00-1 ✓
1	4	0100 ✓		(1,5)	0-01
1	8 (d)	1000 ✓		(1,9)	-001 ✓
2	3	0011 ✓		(4,5)	010-
2	5	0101 ✓		(4,6)	01-0
2	6 (d)	0110 ✓		(8,9)	100- ✓
2	9	1001 ✓		(8,10)	10-0 ✓
2	10	1010 ✓		(3,11)	-011 ✓
3	11	1011 ✓		(9,11)	10-1 ✓
				(10,11)	101- ✓

Step 3 (column II → III):

- `00-1 + 10-1 → -0-1` (1,3,9,11); `-001 + -011 → -0-1` (same term, write once).
- `100- + 101- → 10--` (8,9,10,11); `10-0 + 10-1 → 10--` (same).
- Unticked in column II: `0-01`, `010-`, `01-0`.

Prime implicants: `-0-1 = B'D`, `10-- = AB'`, `0-01 = A'C'D`, `010- = A'BC'`, `01-0 = A'BD'`.

PI chart (columns = required minterms 1,3,4,5,9,10,11):

PI	1	3	4	5	9	10	11
`B'D` (1,3,9,11)	×	⊗			×		×
`AB'` (8,9,10,11)					×	⊗	×
`A'C'D` (1,5)	×			×			
`A'BC'` (4,5)			×	×			
`A'BD'` (4,6)			×				

- Column 3 has only `B'D` → EPI. Column 10 has only `AB'` → EPI.
- Left uncovered: 4 and 5 → `A'BC'` covers both.
- **MSP: `F = B'D + AB' + A'BC'`.**
- ⚠️ EXAM TRICK: never put don't-care columns (6, 8) in the PI chart. Don't-cares are used for merging only.

---

10. NAND-ONLY / NOR-ONLY REALISATION

10.1 SOP → NAND-NAND (2-level)

- Write the minimal SOP.
- Put two bubbles on every wire between the AND level and the OR level (two inversions cancel).
- Each AND + output bubble becomes a NAND; the OR with bubbled inputs is a NAND (De Morgan).
- Same gate count as AND-OR.

```
F = AB + CD
 A ─┐                          A ─┐
    AND─┐                          NAND─┐
 B ─┘   OR── F      ≡         B ─┘     NAND── F
 C ─┐    │                    C ─┐     │
    AND─┘                          NAND─┘
 D ─┘                          D ─┘
```

10.2 POS → NOR-NOR

- The exact dual: each OR becomes a NOR, and the final AND (with bubbled inputs) becomes a NOR.

10.3 The single-literal trap

- `F = AB + C`: the `C` term has no first-level gate, so it must reach the second-level NAND **complemented**: feed `C'` (if complements are free) or add a NAND inverter.
- ⚠️ EXAM TRICK: "Both true and complement inputs are available" means a single literal can be fed straight in, in whichever polarity you need. "Only true inputs" means every complement costs an inverter (1 NAND/NOR each).

10.4 Which form to minimise

- Asked for **NAND** → start from the minimal **SOP**. Asked for **NOR** → start from the minimal **POS** (group the 0s).
- **2-input gates only:** gates with more inputs must be split, so first **factor** the expression to share sub-terms and cut fan-in, then convert.

10.5 Worked: BCD prime detector (Midsem 2024 Q3)

- Z(P,Q,R,S) = 1 for 2, 3, 5, 7; inputs 10–15 are don't-cares; 0 and 1 are not prime.

PQ \ RS	00	01	11	10
00	0	0	1	1
01	0	1	1	0
11	X	X	X	X
10	0	0	X	X

- PIs: `Q'R` (2,3,10,11), `QS` (5,7,13,15), `RS` (3,7,11,15).
- EPIs: `Q'R` (only PI on m2), `QS` (only PI on m5). `RS` is a PI but **not** essential and not needed.
- **Minimum SOP:** **`Z = Q'R + QS`** (unique).
- 2-input NAND realisation (true and complement inputs available): **3 gates**.

```
Q' ─┐
    NAND── n1 ─┐
R  ─┘          NAND── Z = (n1·n2)' = Q'R + QS
Q  ─┐          │
    NAND── n2 ─┘
S  ─┘
```

10.6 Worked: lighthouse controller with 2-input NORs (Midsem 2025 Q1 Task A)

- **Inputs:** W (weather), X Y Z = 3-bit Gray code of the angle sector (000, 001, 011, 010, 110, 111, 101, 100 for 0–45°, 45–90°, ...).
- **Requirement:**
  - W = 0: L is ON for 0–90°, OFF for 90–180°, don't-care for 180–270° (land facing), OFF for 270–360°.
  - W = 1: L alternates every 45° starting ON at 0°.
- **Truth table** (rows in angle order, placed by binary value):

Sector	XYZ (Gray)	Minterm (W=0)	L (W=0)	Minterm (W=1)	L (W=1)
0–45	000	m0	1	m8	1
45–90	001	m1	1	m9	0
90–135	011	m3	0	m11	1
135–180	010	m2	0	m10	0
180–225	110	m6	X	m14	X
225–270	111	m7	X	m15	X
270–315	101	m5	0	m13	1
315–360	100	m4	0	m12	0

- `L = Σm(0,1,8,11,13) + d(6,7,14,15)`.

WX \ YZ	00	01	11	10
00	1	1	0	0
01	0	0	X	X
11	0	1	X	X
10	1	0	1	0

- **Group the 0s (PIs of L'):** `YZ'` (2,6,10,14), `XZ'` (4,6,12,14), `W'Y` (2,3,6,7), `W'X` (4,5,6,7), `WX'Y'Z` (9). All five are essential.
- **Minimal POS:** `L = (Y' + Z)(X' + Z)(W + Y')(W + X')(W' + X + Y + Z')`.
- **Factor before going to 2-input NORs:**
  - `(Y' + Z)(X' + Z) = Z + X'Y'` and `(W + Y')(W + X') = W + X'Y'`.
  - Let `U = X + Y` (so `X'Y' = U'`) and `M = WZ`.
  - `L = (U' + Z)(U' + W)(M' + U) = (U' + M)(M' + U) = U ⊙ M`.
  - **`L = (X + Y) ⊙ (W·Z)`**. Check m9 (W=1, X=Y=0, Z=1): U = 0, M = 1 → L = 0 ✓.
- **2-input NOR circuit (7 gates).** Since `a = U'`, we have `L = U ⊙ M = a ⊕ M`; build XNOR(a, M) with the 4-NOR block from §6.1, then invert it.

```
Netlist (every gate is a 2-input NOR):
  g1:  a = NOR(X , Y )        = (X+Y)'  = U'
  g2:  M = NOR(W', Z')        = WZ
  g3:  t = NOR(a , M )
  g4:  p = NOR(a , t )
  g5:  r = NOR(M , t )
  g6:  q = NOR(p , r )        = a ⊙ M
  g7:  L = NOR(q , q )        = q' = a ⊕ M = U ⊙ M

 X ─┐                     ┌──────────────[g4]─ p ─┐
    [g1]── a ─────────────●──┐            ▲        [g6]─ q ─┬─[g7]── L
 Y ─┘                        [g3]── t ────┤        │        └──┘
 W'─┐                     ┌──┘            ▼        │
    [g2]── M ─────────────●──────────────[g5]─ r ──┘
 Z'─┘
```

- ⚠️ EXAM TRICK: when a POS has many terms, look for a common sub-expression (here `X'Y'`) before converting. A direct NOR-NOR of a 5-term POS with 2-input gates needs far more gates.

---

11. ADDERS

11.1 Half adder (adds two bits, no carry-in)

A	B	C (carry)	S (sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- `S = A ⊕ B`, `C = AB`.

```
A ──┬──────┐
    │     XOR──── S
B ──┼──┬───┘
    │  │
    └──┴──AND──── C
```

11.2 Full adder (adds A, B and a carry-in)

A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

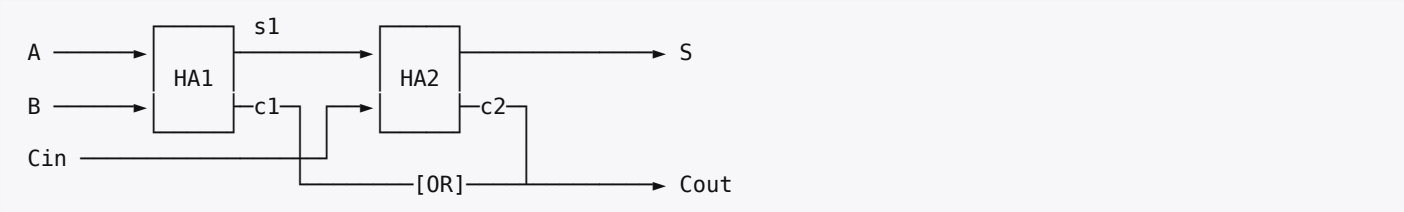
- K-maps:

S: A \ B Cin	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Cout: A \ B Cin	00	01	11	10
0	0	0	1	0
1	0	1	1	1

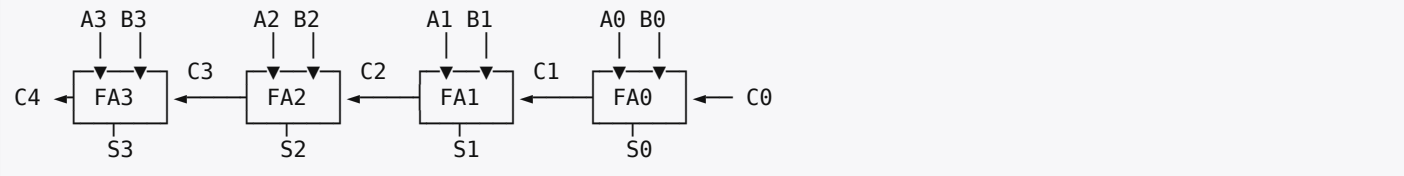
- `S = A ⊕ B ⊕ Cin` (checkerboard, so use XORs). `S = Σm(1,2,4,7)`.
- `Cout = AB + ACin + BCin = AB + Cin(A ⊕ B)` (**majority** function). `Cout = Σm(3,5,6,7)`.

11.3 Full adder from two half adders + OR



- `HA1(A,B) → s1 = A⊕B, c1 = AB`; `HA2(s1, Cin) → S, c2 = (A⊕B)Cin`; `Cout = c1 + c2`.
- c1 and c2 are never 1 together, so the OR could equally be an XOR.

11.4 Ripple-carry adder (n full adders chained)



- **Worst-case delay = `2n + 1` gate levels.** The carry passes through 2 gate levels per stage, plus 1 for the final XOR.
  - 4-bit → 9; 64-bit → **129** gate delays.
- Max clock frequency ≈ `1 / ((2n + 1) · t_pd)`. At 1 ns per gate, a 64-bit RCA allows about 7.75×10^6 additions/s.
- Gate count `O(n)`, delay `O(n)`.
- ⚠️ EXAM TRICK: a 4-bit adder with carry-in has 9 inputs, so its truth table has $2^9 = 512$ rows. That is why we build it from full adders instead of one big K-map.

11.5 Using a full adder as a gate (component-restricted questions)

- Cin = 0: `S = A ⊕ B` (XOR), `Cout = AB` (AND).
- Cin = 1: `S = (A ⊕ B)'` (XNOR), `Cout = A + B` (OR).
- See §23 for the full "blocks as gates" toolkit.

---

12. SUBTRACTORS

12.1 Half subtractor (X − Y)

X	Y	B (borrow)	D (difference)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- `D = X ⊕ Y`, `B = X'Y`.
- ⚠️ EXAM TRICK: the borrow is **not symmetric**: `X'Y ≠ XY'`. Swapping the X and Y ports leaves D correct but makes B wrong. (The adder's carry `AB` is symmetric, which is why students miss this.)

```
X ────────────────XOR──── D
    │            ┌┘
Y ──┼────────────┤
    │            └────────┐
    └──|>o── X'  ──AND──── B = X'Y
```

12.2 Full subtractor (X − Y − Bin)

X	Y	Bin	Bout	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- `D = X ⊕ Y ⊕ Bin = Σm(1,2,4,7)` (same as the FA sum).
- `Bout = X'Y + X'Bin + YBin = X'Y + (X' + Y)Bin = Σm(1,2,3,7)`.

Bout: X \ Y Bin	00	01	11	10
0	0	1	1	1
1	0	0	1	0

12.3 Full subtractor from two half subtractors + OR

```
                   d1
X   ───────►┌────┐──────────►┌────┐
            │x   │           │x   │──────────────────► D
            │ HS1│           │ HS2│  (computes d1 − Bin)
Y   ───────►│y   │─b1─┐  ┌──►│y   │─b2─┐
            └────┘    │  │   └────┘    │
Bin ──────────────────┼──┘             │
                      └─────[OR]───────┴────────► Bout
```

- `b2 = d1'·Bin = (X⊕Y)'Bin`; `Bout = X'Y + (X⊕Y)'Bin = X'Y + X'Bin + YBin` (proof in Tut-3 Q1b).
- ⚠️ EXAM TRICK (Tut-3 Q3b): wiring HS2 as `.x(bin), .y(d1)` computes `Bin − d1`. D is still right (XOR is symmetric) but Bout is wrong. Counter-example: `(X,Y,Bin) = (0,0,1)` gives Bout = 0, but `0 − 0 − 1` needs a borrow (Bout = 1).

12.4 Using a subtractor as a gate

- Half subtractor with X = 1: `D = Y'` (a **free inverter**), B = 0.
- Full subtractor with X = 0: `Bout = Y + Bin` (**OR**). With X = 1: `Bout = Y·Bin` (**AND**).

---

13. 4-BIT ADDER / SUBTRACTOR

```
M (mode) ────────●───────────────●───────────────●───────────────●─────────────┐
                 │               │               │               │             │
     B3 ──[XOR]      B2 ──[XOR]      B1 ──[XOR]      B0 ──[XOR]                 │
                 │               │               │               │             │
       A3        │     A2        │     A1        │     A0        │             │
        │        │      │        │      │        │      │        │             │
        ▼        ▼      ▼        ▼      ▼        ▼      ▼        ▼             │
 C4 ◄── ┌────FA3────┐◄── ┌────FA2────┐◄── ┌────FA1────┐◄── ┌────FA0────┐◄── C0 ──┘
     C4 └─────┬─────┘ C3 └─────┬─────┘ C2 └─────┬─────┘ C1 └─────┬─────┘
              S3               S2               S1               S0

  Overflow (signed):  V = C3 ⊕ C4      (C3 = carry INTO FA3, C4 = carry OUT of FA3)
```

- `M = 0`: each `Bi ⊕ 0 = Bi`, `C0 = 0` → **A + B**.
- `M = 1`: each `Bi ⊕ 1 = Bi'`, `C0 = 1` → **A + B' + 1 = A − B**.
- XOR acts as a programmable inverter controlled by M.
- **Is the subtraction result right as is?**
  - Unsigned, A ≥ B: yes, and C4 = 1 (discard it).
  - Unsigned, A < B: the output is the 2's complement of (B − A), and C4 = 0, meaning a borrow occurred.
  - Signed: correct unless `V = C3 ⊕ C4 = 1`.
- ⚠️ EXAM TRICK: in subtract mode the carry-out is an **inverted borrow**. C4 = 1 means **no** borrow.

---

14. CARRY-LOOKAHEAD ADDER (CLA)

14.1 Generate and propagate

- `Gi = Ai·Bi`: this bit **generates** a carry by itself, whatever the carry-in.
- `Pi = Ai ⊕ Bi`: this bit **propagates** an incoming carry.
- `C(i+1) = Gi + Pi·Ci`, and `Si = Pi ⊕ Ci`.
- Unroll the recursion so every carry depends only on the G's, P's and C0:
  - `C1 = G0 + P0C0`
  - `C2 = G1 + P1G0 + P1P0C0`
  - `C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0`
  - `C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0`

```
A3B3    A2B2    A1B1    A0B0
 │       │       │       │
[PG3]   [PG2]   [PG1]   [PG0]       1 gate level: Gi = AiBi, Pi = Ai⊕Bi
 │G,P    │G,P    │G,P    │G,P
┌▼───────▼───────▼───────▼───┐
│  Carry-lookahead logic      │ ◄── C0      2 levels: flat AND-OR for every Ci
│  (2-level AND-OR per Ci)    │ ──► C4
└┬───────┬───────┬───────┬────┘
 C3      C2      C1      C0
 │       │       │       │
XOR     XOR     XOR     XOR                1 level: Si = Pi ⊕ Ci
 S3      S2      S1      S0
```

- **Delay ≈ 4 gate levels, constant**, whatever n is: G/P (1), AND-OR (2), sum XOR (1).
- ⚠️ EXAM TRICK: for the **carry** alone, `Pi = Ai + Bi` also works (when both are 1, Gi already covers it). For the **sum** `Si = Pi ⊕ Ci`, Pi must be the XOR.

14.2 Scaling (Tut-3 Q4): know these four lines

Design	Gate count	Delay	Comment
Ripple-carry	O(n)	O(n) (`2n+1`)	64-bit = 129 levels
Flat CLA for all n bits	**$O(n^2)$**	**O(1)**	Cn needs n+1 product terms; AND fan-in n+1 is not physically realisable
4-bit CLA blocks chained in ripple	O(n)	O(n) (n/4 × const)	smaller slope, same growth
Hierarchical lookahead (block G/P)	O(n)	**O(log n)**	64-bit, 4-bit groups: $\log_4 64$ = 3 levels → about 12 gate delays

- Block signals for a 4-bit group:
  - `G_blk = G3 + P3G2 + P3P2G1 + P3P2P1G0` (the block makes a carry by itself).
  - `P_blk = P3P2P1P0` (an incoming carry passes straight through the block).

14.3 Borrow-lookahead (Tut-3 Q1c,d)

- From `Bout = X'Y + (X' + Y)Bin`:
  - Borrow-generate `Gi = Xi'Yi`; borrow-propagate `Pi = Xi' + Yi` (**OR form**: `Xi ⊕ Yi` does **not** work here).
- `B4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0·Bin`.

---

15. BCD (DECIMAL) ADDER

- Add two BCD digits and a carry-in with an ordinary 4-bit binary adder, giving `K Z8 Z4 Z2 Z1` (K = binary carry-out).
- If the binary sum is **9 or less**, it is already the BCD answer, with decimal carry 0.
- If it is **more than 9** (10–19), add `0110` (6) to skip the six unused codes, and set the decimal carry.
- **Correction condition:** `C = K + Z8Z4 + Z8Z2` (the K-map of "Z8Z4Z2Z1 > 9" gives `Z8Z4 + Z8Z2`).

```
    A3..A0   B3..B0
      |4       |4
   ┌──▼────────▼─┐
   │ 4-bit adder │◄── Cin
   └─┬──┬──┬──┬──┘
 K   Z8 Z4 Z2 Z1
 │    │  │  │  │
 │  ┌─┴──┴──┴──┴───────────┐
 └─►│ C = K + Z8Z4 + Z8Z2  ├──────────────► Cout (decimal carry)
    └──────────┬───────────┘
               │C
      0  C  C  0  (adds 0110 when C = 1)
   ┌──▼──▼──▼──▼──────────────┐
   │ 4-bit adder (Z8 Z4 Z2 Z1 +) │──► carry ignored
   └─┬──┬──┬──┬──────────────┘
     S8 S4 S2 S1    (BCD digit)
```

- Example: `8 + 7 = 15` → binary `0 1111` → C = Z8Z4 = 1 → `1111 + 0110 = 1 0101` → digit `0101` = 5, Cout = 1 → **15** ✓.
- Example: `9 + 9 + 1 = 19` → binary `1 0011` (K = 1) → `0011 + 0110 = 1001` = 9, Cout = 1 → **19** ✓.
- An n-digit BCD adder is n such stages, decimal carry to decimal carry.

---

16. CODE CONVERTERS

16.1 General procedure (any multi-output circuit)

- Count the inputs and outputs.
- Write **one** truth table with all outputs; mark impossible inputs as X.
- Draw **one K-map per output**; share gates between outputs where possible.

16.2 BCD → Excess-3 (inputs A B C D, outputs w x y z)

Dec	A B C D	w x y z
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100
10–15	—	X X X X

- `w = A + BC + BD = A + B(C + D)`
- `x = B'C + B'D + BC'D' = B'(C + D) + B(C + D)' = B ⊕ (C + D)`
- `y = CD + C'D' = (C ⊕ D)'`
- `z = D'`
- The `(C + D)` sub-term is shared between w and x.

16.3 BCD → 7-segment decoder

```
  —a—        digit │ a b c d e f g
 f   b       ──────┼──────────────
  —g—          0   │ 1 1 1 1 1 1 0
 e   c         1   │ 0 1 1 0 0 0 0
  —d—          2   │ 1 1 0 1 1 0 1
               3   │ 1 1 1 1 0 0 1
               4   │ 0 1 1 0 0 1 1
               5   │ 1 0 1 1 0 1 1
               6   │ 1 0 1 1 1 1 1
               7   │ 1 1 1 0 0 0 0
               8   │ 1 1 1 1 1 1 1
               9   │ 1 1 1 1 0 1 1
```

- 4 inputs, 7 outputs, don't-cares 10–15 → 7 K-maps. E.g. `a = Σm(0,2,3,5,6,7,8,9) + d(10–15) = A + C + BD + B'D'`.
- Common-cathode display = active-high outputs; common-anode = active-low (a segment lights when its line is 0).
- Multiplexed multi-digit displays: Tut-5 Q3.3.

---

17. MAGNITUDE COMPARATOR

- **1-bit:** `(A>B) = AB'`, `(A<B) = A'B`, `(A=B) = (A ⊕ B)' = A'B' + AB`.

```
A ──┬──────────┐             A ─┐
    │          AND── A>B        XNOR── A=B
B'──┼──────────┘             B ─┘
A'──┴──────────┐
               AND── A<B
B ─────────────┘
```

- **n-bit:** let `xi = (Ai ⊕ Bi)'` (bit i equal). Compare in dictionary order starting at the MSB.
  - `(A = B) = x3x2x1x0`
  - `(A > B) = A3B3' + x3A2B2' + x3x2A1B1' + x3x2x1A0B0'`
  - `(A < B) = A3'B3 + x3A2'B2 + x3x2A1'B1 + x3x2x1A0'B0`
- Reading a term: `x3x2A1B1'` = "bits 3 and 2 are equal, and at bit 1 A has 1 while B has 0".
- 1-bit comparator from one 2-to-4 decoder + one OR (Tut-5 Q1a.1): `X>Y = D2`, `X=Y = D0 + D3`.

---

18. BINARY MULTIPLIERS

- A **partial product** is the multiplicand ANDed with one multiplier bit, shifted left by that bit's position. Add the partial products.
- **2 × 2 multiplier** (A1A0 × B1B0):

```
                A1      A0
           ×    B1      B0
         ───────────────────
               A1B0    A0B0
        A1B1   A0B1
  ──────────────────────────
   C3     C2      C1      C0

C0 = A0B0
C1 = HA1.S  of (A1B0, A0B1)
C2 = HA2.S  of (A1B1, HA1.C)
C3 = HA2.C
Hardware: 4 AND gates + 2 half adders.
```

- **J-bit × K-bit:** `J·K` AND gates, `(J − 1)` K-bit adders, product is `J + K` bits.
- **Multiply by a constant:** decompose into shifts: `6X = 4X + 2X = (X << 2) + (X << 1)`. One adder per extra 1 bit in the constant.

---

19. DECODERS

19.1 What a decoder does

- n-bit input code → `2^n` output lines, **exactly one** active at a time.
- **Each active-high output is a minterm:** `Dk = mk`.
- Built from `2^n` AND gates (n inputs each, n+1 with enable) plus n inverters.

```
2-to-4 decoder with enable (active-high)
A ─────|>o─ A'                        EN  A  B │  D0  D1  D2  D3
B ──┼──┼───|>o─ B'                    ─────────┼────────────────
    │  │                               0   x  x │  0   0   0   0
D0 = EN·A'B'                           1   0  0 │  1   0   0   0
D1 = EN·A'B                            1   0  1 │  0   1   0   0
D2 = EN·AB'                            1   1  0 │  0   0   1   0
D3 = EN·AB                             1   1  1 │  0   0   0   1
```

- **Enable:** EN = 0 forces every output inactive. A decoder with enable **is** a demultiplexer (data on EN).

19.2 Active-high vs active-low

- **Active-low** (NAND-based, bubbles on the outputs): the selected output is 0, all others 1.
  - Each active-low output is a **maxterm**: `Dk' = (mk)' = Mk`.

19.3 Implementing any function with one decoder: pick the cheapest of four

Decoder outputs	Gate	Connect outputs for…	Gives
Active-high (`mk`)	OR	rows where F = 1	`Σm`
Active-high (`mk`)	NOR	rows where F = 0	`(Σm of 0s)' = F`
Active-low (`Mk`)	NAND	rows where F = 1	`(ΠMk)' = Σmk = F`
Active-low (`Mk`)	AND	rows where F = 0	`ΠM = F`

- Gate fan-in = min(#1s, #0s) if you choose cleverly.
- One decoder can serve several functions at once (one OR per function), e.g. full adder: `S = Σm(1,2,4,7)`, `C = Σm(3,5,6,7)` from one 3-to-8 decoder and two 4-input ORs.
- A function that is a **single** minterm (or maxterm) needs **no gate**: just use that output (or invert it).
- ⚠️ EXAM TRICK (Tut-5 Q1b.1): `H = B(A+B+C')(A'+B'+C)` = ΠM(0,1,4,5,6) = Σm(2,3,7). With active-low outputs, NAND(D2', D3', D7') uses a **3-input** gate, while AND of the 0-rows needs **5** inputs. Always check both.

19.4 Hierarchical decoders

- 3-to-8 from two 2-to-4: B and C go to both decoders; `A'` enables the upper one (D0–D3), `A` enables the lower one (D4–D7).
- 4-to-16 from five 2-to-4: one decoder on the two MSBs drives the four enables of four decoders on the two LSBs.

```
                 ┌──────────┐  D0
          B  ───►│  2-to-4  │  ..
          C  ───►│          │  D3
   A'  ─────────►│EN        │
                 └──────────┘
                 ┌──────────┐  D4
          B  ───►│  2-to-4  │  ..
          C  ───►│          │  D7
   A   ─────────►│EN        │
                 └──────────┘
```

- ⚠️ EXAM TRICK: the **MSB** variable drives the enables. If you swap it, the outputs come out in the wrong order.
- ⚠️ EXAM TRICK (Lec-13 review): "4 inputs, 6 outputs, minimum components" → **1** 4-to-16 decoder + **6** OR gates (one per output), **0** extra AND gates.

---

20. ENCODERS & PRIORITY ENCODERS

20.1 Plain encoder (2^n inputs → n-bit index)

- An 8-to-3 encoder is just OR gates:
  - `Y2 = D4 + D5 + D6 + D7`
  - `Y1 = D2 + D3 + D6 + D7`
  - `Y0 = D1 + D3 + D5 + D7`
- **Two flaws:**
  - Two inputs active at once → the output is the OR of both codes (garbage, e.g. D3 + D4 → 111).
  - "No input active" and "D0 active" both give 000.

20.2 Priority encoder (4-to-2)

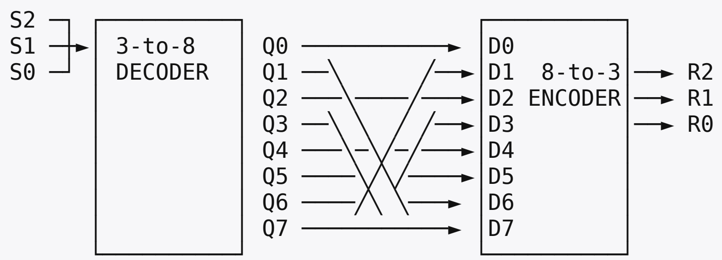
D3	D2	D1	D0	x	y	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

- The highest-index active input wins; V (valid) = 1 if any input is active.
- `x = D2 + D3`, `y = D3 + D1D2'`, `V = D0 + D1 + D2 + D3`.
- ⚠️ EXAM TRICK: in `y = D3 + D1D2'`, the `D2'` matters: D1 counts only if the higher-priority D2 is off.

20.3 Decoder → wires → encoder = any one-to-one code converter

- Decode the input code to one active line k; wire line k to encoder input `f(k)`; the encoder outputs the new code.
- The mapping `f` must be one-to-one (a permutation).
- **Binary → Gray (Tut-5 Q3.1):** `0→0, 1→1, 2→3, 3→2, 4→6, 5→7, 6→5, 7→4`.
- **Bit reversal (Quiz 2 Q4):** `0→0, 1→4, 2→2, 3→6, 4→1, 5→5, 6→3, 7→7`.

```
Quiz 2 Q4: bit reversal (received b2b1b0 → corrected b0b1b2)
```




```
Wiring list (this is what gets marked):
  Horizontal (palindromes, unchanged):  Q0→D0, Q2→D2, Q5→D5, Q7→D7
  Diagonal (swapped pairs):             Q1→D4, Q4→D1, Q3→D6, Q6→D3
```

21. MULTIPLEXERS (DATA SELECTORS)

21.1 Structure

- `2^n` data inputs, n select lines, 1 output: `Y = Σ mk(S)·Ik`.
- 2:1: `Y = S'I0 + SI1`.
- 4:1: `Y = S1'S0'I0 + S1'S0I1 + S1S0'I2 + S1S0I3`.

- Internally a decoder on the select lines whose outputs gate the data inputs, followed by an OR.

```
4-to-1 MUX                          S1 S0 | Y
I0 ──AND(S1'S0')─┐                  ──────┼───
I1 ──AND(S1'S0 )─┤                  0  0  | I0
I2 ──AND(S1 S0')─OR── Y             0  1  | I1
I3 ──AND(S1 S0 )─┘                  1  0  | I2
                                    1  1  | I3
```

- **Enable:** a bubble on EN means active-low: EN = 0 → the mux works normally.
  - When disabled, the output is forced to a fixed value, independent of the data inputs.
  - The Lec-13-14 slide's 4-to-1 mux outputs **1** when disabled; the textbook's mux (and most ICs) output **0**.
- ⚠️ EXAM TRICK: always use the disabled-output value given in the question's function table. In hierarchical designs that OR or AND mux outputs together, getting it wrong breaks the circuit.
- **Dual 4-to-1 mux:** selects one 2-bit word out of four with shared select lines. Used to produce sum and carry of a full adder in one device.
- **8:1 from two 4:1 + one 2:1:** low select bits drive both 4:1s; the MSB select picks between them.

21.2 Implementing functions with a mux: three methods

Method 1: all n variables on the select lines. Tie each `Ik` to the truth-table output of row k (0 or 1).

- Tut-5 Q2a.2: `F = Σm(0,2,5,7)` on an 8:1 → `I0..I7 = 1,0,1,0,0,1,0,1`.

Method 2: n−1 variables on selects, one variable (say D) left over. Each data input is one of `{0, 1, D, D'}`.

- Implementation table: list the minterms with D = 0 in one row and D = 1 in the other; circle the 1s; read each column.
- Lecture example `F(A,B,C,D) = Σm(0,1,3,4,8,9,15)`, selects A B C:

ABC (select)	000	001	010	011	100	101	110	111
D = 0	**m0**	m2	**m4**	m6	**m8**	m10	m12	m14
D = 1	**m1**	**m3**	m5	m7	**m9**	m11	m13	**m15**
Ik	1	D	D'	0	1	0	0	D

- Rule: both circled → 1; neither → 0; only the D=1 cell → D; only the D=0 cell → D'.

Method 3: n−2 variables on selects (4:1 mux for a 4-variable function). Each Ik is a function of the 2 leftover variables: take that K-map row and simplify it (use Xs).

- Quiz 2 Q3 (F from §8.6, A B on selects):

AB	CD cells (00,01,11,10)	Ik
00	0, 0, X, 1	`C`
01	1, 0, 1, 1	`C + D'`
10	0, 0, 1, 0	`CD`
11	1, 0, 0, X	`D'`

- Note the mux row order: I2 is AB = 10, I3 is AB = 11 (the K-map draws row 11 before row 10).

21.3 Cascading muxes

- Full-adder sum with two 4:1 muxes (Tut-5 Q2d): mux 1 (selects X, Y; data 0,1,1,0) = `X ⊕ Y`; mux 2 (selects X⊕Y, Cin; data 0,1,1,0) = `S`.

21.4 Only true inputs available (Midsem 2024 Q2): the decoder trick

- If complements are not available and a data input needs `D'` or a function of the leftover variables, feed the leftover variables into a **decoder**. It gives every minterm of them for free, including complemented ones.
- Full worked solution in §26.4.
- ⚠️ EXAM TRICK: "Use AB as select input (s1 s0) respectively" means **A = s1 (MSB)**. Getting the order wrong permutes I1 and I2 and loses all the marks.

---

22. DEMULTIPLEXERS

- Route one data line D to one of `2^n` outputs chosen by the selects: `Yk = D·mk(S)`.

```
                  ┌────────┐ ── Y0 = D·S1'S0'
D ──────────────► │ 1-to-4 │ ── Y1 = D·S1'S0
                  │ DEMUX  │ ── Y2 = D·S1S0'
                  └────────┘ ── Y3 = D·S1S0
                     |   |
                    S1   S0
```

- Exactly a decoder with D wired to its enable. A 2-to-4 decoder with EN serves as a 1-to-4 demux.
- Uses: the receiving end of a time-multiplexed link; enabling one of n digits/registers (Tut-5 Q3.3).
- ⚠️ EXAM TRICK: a demux whose **select lines are driven by other signals** computes products of their complements: with `S1 = a`, `S0 = b`, data = 1, `Y0 = a'b'`. Midsem 2024 Q2 exploits this (§26.4).

---

23. USING ARITHMETIC BLOCKS AS GATES (component-restricted designs)

Block	Tie-off	Output	Function
Half adder	—	S / C	`A ⊕ B` / `AB`
Full adder	Cin = 0	S / Cout	`A ⊕ B` / `AB`
Full adder	Cin = 1	S / Cout	`(A ⊕ B)'` / `A + B`
Half subtractor	X = 1	D	`Y'` (inverter)
Half subtractor	—	B	`X'Y` (AND with one inverted input)
Full subtractor	X = 0	Bout	`Y + Bin` (OR)
Full subtractor	X = 1	Bout	`Y·Bin` (AND)
Full adder	—	Cout	majority(A, B, C)

- ⚠️ EXAM TRICK: logic 0 and logic 1 (GND/VCC) are always allowed even when "only true inputs are available". Tying a pin to a constant turns a block into a gate.

- Full worked example (Midsem 2024 Q4) in §26.5.

---

24. PROGRAMMABLE LOGIC DEVICES (ROM / PLA / PAL)

- Every SOP circuit is an **AND array** (products) feeding an **OR array** (sums). A PLD pre-builds both arrays with programmable crossings (× = connection).

Device	AND array	OR array	Think of it as
ROM	fixed (full decoder: all `2^n` minterms)	programmable	a stored truth table, size `2^n × m`
PLA	programmable	programmable	shared minimal SOP
PAL	programmable	fixed (each output owns its ANDs)	separate minimal SOPs

```
PLA (3 inputs, 4 product terms, 2 outputs)
   A   A'   B   B'   C   C'
   |   |    |   |    |   |
 ──×───┼────×───┼────┼───┼──  P1  ───×──────┼───
 ──┼───×────┼───┼────×───┼──  P2  ───┼──────×───
 ──×───┼────┼───┼────┼───×──  P3  ───×──────×───
 ──┼───×────┼───×────┼───┼──  P4  ───┼──────×───
            AND array                F1     F2    (OR array)
```

- **PLA size = (#inputs) × (#product terms) × (#outputs).** "Minimum-size PLA" means **fewest distinct product terms** across all outputs.
  - A term needed by two outputs is built once and shared. It can pay to use a **non-prime** term (even a full minterm) if it can be shared.
- **PAL:** no sharing; minimise each output separately and check the per-output product-term limit.
- **Programming table:** one row per product term. Input columns: `1` = true literal, `0` = complemented literal, `-` = variable not used. Output columns: `1` = term connected to that output's OR (`-` or blank = not connected).

24.1 Worked: full subtractor on a minimum PLA (Midsem 2025 Q2)

- Inputs P (MSB), Q, R; `D = Σm(1,2,4,7)`, `B = Σm(1,2,3,7)`.
- D is a checkerboard: it needs all 4 minterms (`P'Q'R, P'QR', PQ'R', PQR`).
- B's minimal SOP (`P'Q + P'R + QR`) shares nothing with D → 4 + 3 = 7 terms.
- **Better:** reuse D's minterms in B. B needs m1, m2, m3, m7: m1 = `P'Q'R` ✓, m2 = `P'QR'` ✓, m7 = `PQR` ✓ are already built; only m3 needs one extra term, e.g. `P'Q` (covers m2, m3).
- **5 product terms** → PLA size `3 × 5 × 2`.

#	Product term	P	Q	R	B	D
1	`P'Q'R`	0	0	1	1	1
2	`P'QR'`	0	1	0	1	1
3	`PQ'R'`	1	0	0	–	1
4	`PQR`	1	1	1	1	1
5	`P'Q`	0	1	–	1	–

- `B = P'Q'R + P'QR' + PQR + P'Q` (= Σm(1,2,3,7) ✓). Term 5 could equally be `QR` or `P'R`.

24.2 Worked: minimum PLA (Comprehensive 2024 Q6)

- `F1 = Σm(1,3,5,7,8,10,11)`, `F2 = Σm(6,7,8,10,11,14,15)`.
- Separate MSPs: `F1 = A'D + AB'D' + AB'C` (or `A'D + B'CD + AB'D'`); `F2 = BC + AC + AB'D'`.
- Share: choose `F1 = A'D + AB'D' + AB'C` and `F2 = BC + AB'D' + AB'C` (AB'C replaces AC: it covers 10, 11, and 14, 15 are still covered by BC).
- **4 distinct terms:** `A'D`, `AB'D'`, `AB'C`, `BC` → size `4 × 4 × 2`.
- Lower bound: F1 needs ≥ 3 terms, and F2 needs a term over m6, which is not in F1. So 4 is the minimum.

Term	A	B	C	D	F1	F2
`A'D`	0	–	–	1	1	–
`AB'D'`	1	0	–	0	1	1
`AB'C`	1	0	1	–	1	1
`BC`	–	1	1	–	–	1

---

25. VERILOG ESSENTIALS (combinational)

25.1 Structural (gate-level)

- Gate primitives list the **output first**: `and (y, a, b);`, `xor (s, a, b);`, `not (yn, a);`.
- Statements describe hardware wired in parallel; **line order does not matter**. Tut-3 Q3a uses `w1` before the line that drives it, which is legal.

```
module half_adder(input a, b, output s, c);
  xor (s, a, b);
  and (c, a, b);
endmodule

module full_adder(input a, b, cin, output s, cout);
  wire s1, c1, c2;
  half_adder HA1(.a(a),  .b(b),   .s(s1), .c(c1));   // named ports: safe
  half_adder HA2(.a(s1), .b(cin), .s(s),  .c(c2));
  or (cout, c1, c2);
endmodule
```

- **Named ports** `.port(signal)` are order-independent. **Positional ports** must match the declaration order exactly.
- ⚠️ EXAM TRICK: for non-symmetric blocks (subtractors), a swapped connection compiles fine and silently gives wrong outputs for some inputs. Always check which port is the minuend.

25.2 Dataflow

- `assign` = continuous assignment; the left side must be a **net** (`wire`).
- `assign y = s ? i1 : i0;` → 2:1 mux. `assign {cout, sum} = a + b + cin;` → concatenation captures the carry.
- Delays: `assign #10 out = a & b;` or `wire #10 out = a & b;`.
- Operator precedence (high → low): unary `+ - ! ~` → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== != === !==` → reduction/bitwise `& ~& ^ ^~ | ~|` → logical `&& ||` → conditional `?:`.
- `~a` is bitwise NOT; `!a` is logical NOT. `&a` is reduction AND (all bits); `^a` is reduction XOR (parity).
- Arithmetic wraps to the destination width (4-bit `1111 + 1 = 0000`).

25.3 Behavioural

- `initial` runs once at time 0. `always @(...)` re-runs every time its sensitivity list fires. They cannot be nested, and all of them run in parallel.
- Combinational logic: `always @(*)` with blocking `=`. Assign every output on every path, or a **latch** is inferred.
- Sequential logic: `always @(posedge clk)` with non-blocking `<=`.
- **Blocking `=`:** updates immediately, like software; the next line sees the new value.
- **Non-blocking `<=`:** evaluate all right-hand sides first, then update all at once at the end of the time step. This models flip-flops sampling together.

25.4 `wire` vs `reg` (Tut-3 Q3c)

- **`wire` (net):** a physical connection with no storage, driven continuously by a gate output, an `assign`, or a module output port. It **cannot** be assigned inside `initial`/`always`.
- **`reg`:** holds its last value until it is assigned again; used for anything assigned in `initial`/`always`. It does **not** necessarily become a hardware register.
- **Testbench rule:** signals the testbench drives (DUT inputs) are `reg`; signals the DUT drives (DUT outputs) are `wire`.
- Other types: `integer` (signed 32-bit), `time`, `real`, vectors `wire [3:0] A;`, memories `reg [7:0] mem [0:15];`, `parameter`/`localparam` constants.

25.5 System tasks

- `$display` prints once, when executed. `$monitor` prints whenever any argument changes (at most once per time step). `$time` = current simulation time. `$stop` pauses (interactive); `$finish` ends the simulation. `#n` waits n time units.

```
module tb_full_adder;
  reg  a, b, cin;           // driven here → reg
  wire s, cout;             // driven by DUT → wire
  full_adder DUT(.a(a), .b(b), .cin(cin), .s(s), .cout(cout));
  initial begin
    $monitor("t=%0t a=%b b=%b cin=%b -> s=%b cout=%b", $time, a, b, cin, s, cout);
    a=0; b=0; cin=0;
    #5 a=1;
    #5 b=1;
    #5 cin=1;
    #5 $finish;
  end
endmodule
```

---

26. PAST-PAPER COMBINATIONAL PROBLEMS, WORKED

> These are the exact question styles the examiners reuse. Each one is solved step by step, with the trick that decides the marks.

26.1 Quiz 2 (Sept 2026), summary of answers

Q	Topic	Set A answer	Set B answer
1	Scrambled truth table → K-map → MSP	`F = A' + BC'`	`F = A' + BC`
2	PIs, EPIs, MSP with Xs	`BD' + A'C + B'CD` (all 3 EPI)	`BD' + A'C + AC'D'` (all 3 EPI)
3	4:1 mux, AB on selects	I0= `C`, I1= `C+D'`, I2= `CD`, I3= `D'`	I0= `C`, I1= `C+D'`, I2= `C'D'`, I3= `D'`
4	Decoder→encoder bit reversal	Q1↔D4, Q3↔D6, Q4↔D1, Q6↔D3, rest straight	same
5	Bit-width detective (unsigned)	wrong rows: 50+14, 45+25; **n = 6**	wrong rows: 20+12, 15+25; **n = 5**

- **Q3 Set B check:** row AB = 10 is `1 0 0 0` → only CD = 00 → `C'D'`.
- **Q3 marking:** 1 mark per line, all-or-none per line. A data input that is correct but not minimal (e.g. `CD' + CD` instead of `C`) risks losing the line.

26.2 Midsem 2025-26 Q1 Task A (lighthouse): POS + 2-input NOR

- Fully solved in §10.6. Key results:
  - `L = Σm(0,1,8,11,13) + d(6,7,14,15)`.
  - PIs of L': `YZ', XZ', W'Y, W'X, WX'Y'Z` (all essential).
  - `L = (Y'+Z)(X'+Z)(W+Y')(W+X')(W'+X+Y+Z')`.
  - Factored: `L = (X+Y) ⊙ (WZ)` → 7 two-input NOR gates.
- ⚠️ EXAM TRICK: the Gray-code sectors run 000, 001, 011, 010, 110, 111, 101, 100. The land-facing sectors (180–270°) are codes `110` and `111` → **m6, m7, m14, m15** are the don't-cares, not m4–m7.

26.3 Midsem 2025-26 Q2: full subtractor on a minimum PLA

- Fully solved in §24.1: **5 product terms**, size 3 × 5 × 2.

26.4 Midsem 2024-25 Q2: mux + decoder/encoder/demux, only TRUE inputs

- **Problem:** `F(A,B,C,D) = Σm(0,2,3,5,6,8,9) + d(10–15)`, A B on the mux selects (A = s1). Only TRUE inputs. Available: one 2-to-4 decoder, one 4-to-2 encoder, one 4:1 mux, one 1:4 demux (all with active-high enable).
- **Step 1: what each data input must be** (functions of C, D):

AB	m (CD = 00, 01, 10, 11)	F values	Ik needed
00 → I0	m0, m1, m2, m3	1, 0, 1, 1	`C + D'` = NOT(C'D)
01 → I1	m4, m5, m6, m7	0, 1, 1, 0	`C ⊕ D`
10 → I2	m8, m9, m10, m11	1, 1, X, X	`1`
11 → I3	m12...m15	X, X, X, X	`1` (or anything)

- **Step 2: get `C + D'` and `C ⊕ D` without complements or gates.**
  - Decoder on (C, D): outputs `d0 = C'D'`, `d1 = C'D`, `d2 = CD'`, `d3 = CD` (complements for free).
  - **I0 = `C + D'` = `C + C'D'`**: use the encoder as an OR gate. Plain encoder: `Y1 = E2 + E3`. Connect `E2 = C` (true input), `E3 = d0`. These two are never 1 together, so even a priority encoder works.

- **I1 = `C ⊕ D = (C'D')'·(CD)'`**: use the demux as a NOR gate. Selects `s1 = d0`, `s0 = d3`, data = 1: `Y0 = 1·s1'·s0' = (C'D')'(CD)' = C ⊕ D`.
- I2 = I3 = logic 1.

```
Connection list
───────────────
DECODER   : inputs (C, D), EN = 1  → d0 = C'D', d1 = C'D, d2 = CD', d3 = CD
ENCODER   : E3 = d0, E2 = C, E1 = 0, E0 = 0, EN = 1  → Y1 = C + C'D' = C + D'
DEMUX     : data = 1, s1 = d0, s0 = d3, EN = 1        → Y0 = d0'·d3' = C ⊕ D
MUX       : s1 = A, s0 = B, I0 = ENC.Y1, I1 = DMX.Y0, I2 = 1, I3 = 1  → F

                    d0  ──────●───────────────────→ E3  ┌─────────┐
  C ──→ ┌──────────┐          │             C ─────→ E2 │ 4-to-2  │ Y1 ─┐
  D ──→ │ 2-to-4   │          │                         │ ENCODER │     │ C+D'
        │ DECODER  │          │                         └─────────┘     │
        └──────────┘  d3 ──┐  │                                         │
                           │  └──→ ┌──────────┐                         │
                           └─────→ │ s1       │  Y0 ──┐                 │
                                   │ s0 1-to-4│       │ C⊕D             │
                 1 ─ data ───────→ │ DEMUX    │       │                 │
                                   └──────────┘       ▼                 ▼
                          ┌────────────────────────── I1 ────────────── I0 ──┐
                   1 ───→ I2          4-to-1 MUX                             ├──→ F
                   1 ───→ I3      s1 = A          s0 = B                     │
                          └──────────────────────────────────────────────────┘
```

- Verified for all 10 specified minterms.
- ⚠️ EXAM TRICK: a demux/decoder output whose **select lines are other signals** is a **product of complements**. With mutually exclusive inputs (minterm lines), `(mi)'(mj)'` = OR of the remaining minterms. That is how you get an OR/XOR with no gate.

26.5 Midsem 2024-25 Q4: one FA, one FS, one HS, only true inputs

- **Problem:** `F(w,x,y,z) = Σm(2,3,4,5,6,8,9) + d(10–15)`.
- **Step 1: K-map / MSP.** PIs: `w`, `x'y`, `xy'`, `yz'`, `xz'`. EPIs: `w`, `x'y`, `xy'`.
  - `F = w + x'y + xy' + yz'` (or `+ xz'`) = `w + (x ⊕ y) + yz'`.
- **Step 2: rewrite F for the available blocks.**
  - For w = 0: `F = (x ⊕ y) + xyz'` (m6 = 0110 is the only 1 not covered by x⊕y).
  - For w = 1: only m8, m9 (x = y = 0) are required to be 1.
  - A full adder on (w, x, y) gives `S = w ⊕ x ⊕ y` and `C = maj(w,x,y)`. For w = 0, S = x ⊕ y and C = xy.
  - So `F = S + z'·C`. Check w = 1, x = y = 0: S = 1 ✓.
- **Step 3: map onto the blocks.**
  - **FA(w, x, y)** → S, C.
  - **HS(X = z, Y = C)** → borrow `B = X'Y = z'·C` (the HS borrow is an AND with one inverted input, so z' costs nothing).
  - **FS(X = 0, Y = S, Bin = z'C)** → `Bout = X'Y + X'Bin + Y·Bin = S + z'C` (FS with X = 0 is an OR gate).
  - **F = FS.Bout**.

```
 w ──→ ┌─────┐  S = w⊕x⊕y ───────────────────────────────→ ┌─────────────┐
 x ──→ │ FA  │                                              │ FS          │
 y ──→ │     │  C = maj(w,x,y) ──→ ┌──────────┐             │ X = 0 (GND) │
       └─────┘                     │ HS       │ B = z'·C ──→│ Y = S       │
                         z ──────→ │ X = z    │             │ Bin=z'C     │
                                   │ Y = C    │             │             │──→ Bout = S + z'C = F
                                   └──────────┘             └─────────────┘
```

- Verified against all 10 specified minterms.

- ⚠️ EXAM TRICK: first write the MSP, then ask "which output of which block already contains a piece of it?" FA sum = 3-input XOR; FA carry = majority; HS borrow = `X'Y` (a free complement); FS borrow with X = 0 = OR, with X = 1 = AND.

26.6 Midsem 2024-25 Q3: BCD prime detector → 2-input NANDs

- Fully solved in §10.5: `Z = Q'R + QS`, EPIs `Q'R` and `QS`, extra PI `RS` (not essential), 3 NAND gates.

26.7 Comprehensive 2025 Q2: 8:1 mux, true + complement inputs, no gates

- `F(A,B,C,D) = Σm(0,1,2,5,8,9,10,11,13,14,15)`, selects A B C, D on the data lines:

ABC	000	001	010	011	100	101	110	111
D=0	**0**	**2**	4	6	**8**	**10**	12	**14**
D=1	**1**	3	**5**	7	**9**	**11**	**13**	**15**
Ik	1	D'	D	0	1	1	D	1

26.8 Comprehensive 2024 Q3: active-low decoder + 2-input NANDs

- `F(a,b,c) = Σm(0,2,5,7)`; the decoder has active-low outputs and an active-low enable (tie EN' = 0).
- Active-low outputs are `Mk = mk'`. `F = m0 + m2 + m5 + m7 = (m0'·m2'·m5'·m7')'` = 4-input NAND of `Y0', Y2', Y5', Y7'`.
- With **2-input NANDs only**: `(abcd)' = NAND(AND(a,b), AND(c,d))`, and each AND = NAND + NAND-inverter.
  - `n1 = NAND(Y0',Y2')`, `n2 = NAND(Y5',Y7')` → `n1 = m0 + m2`, `n2 = m5 + m7`.
  - `F = n1 + n2 = NAND(n1', n2')` → needs `n1'`, `n2'` (2 NAND inverters) → **5 NAND gates**.
  - Alternative with the same count: note `F = a'c' + ac = (a ⊕ c)'`, which also needs 5 NANDs. Either way 5.

---

27. EXAM-DAY MASTER CHECKLIST (combinational)

- **Before any K-map:**
  - Write the truth table **in binary order**, even if the question lists rows in another order (Quiz 2 Q1) or in Gray order (Midsem 2025, Tut-4 Q2).
  - Mark every don't-care: BCD 10–15, unused states, impossible inputs, "downstream ignores it".
- **Drawing the K-map:**
  - Label both axes with the variable names; use the Gray order 00, 01, **11, 10**.
  - Re-check which letter is the MSB.
- **Reading the K-map:**
  - List all PIs first, then EPIs, then the cover. With don't-cares, check for multiple minimal answers and list them all if asked.
  - SOP from the 1s, POS from the 0s, plus De Morgan.
- **Component-restricted designs:**
  - First write what each allowed block computes (minterms, maxterms, majority, XOR, OR via tie-off, etc.), then map the pieces of F onto them.

    - Watch for enable bubbles (active-low), output bubbles (maxterms), and the select order (`s1 s0` = first-named variable is the MSB).
    - "Only true inputs" → complements come from a decoder, an HS/FS borrow, or a NAND/NOR inverter.
- **Arithmetic:**
    - Unsigned overflow = carry-out; signed overflow = `Cn−1 ⊕ Cn`; subtract carry-out = NOT borrow.
    - Bit-width detective: lower bound from correct rows, modulus from wrong rows; check every row.
- **Verilog:**
    - Testbench: DUT inputs `reg`, DUT outputs `wire`.
    - Gate primitives: output first.
    - Positional port order matters; non-symmetric blocks (subtractor) expose wiring bugs.
- **Marking:** most answers are **all-or-none**. Spend 20 seconds re-checking one random K-map cell and one random wire before moving on.

End of Section 1. Section 2 (Tutorial Solutions, Tut-3 → Tut-7) is in `02_Tutorial_Solutions.md`.